COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

PACKAGE 07 / VOLUME 06

# Gate F: Game Capstone

## Undergraduate software release and review

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

# Contents

**Gate F: Undergraduate Software Capstone** 3

Release a small original game on your engine . . . 3

A bounded example: Signal Relay . . . 3

Required system contract . . . 3

Acceptance matrix . . . 4

Milestone 1: scope and vertical slice . . . 5

Milestone 2: simulation and motion . . . 5

Milestone 3: feedback, AI and persistence . . . 5

Milestone 4: network and service evidence . . . 6

Milestone 5: release candidate and review . . . 6

Required failure matrix . . . 7

Evidence directory and provenance . . . 7

Assessment rubric . . . 8

Oral review prompts . . . 9

Postmortem structure . . . 9

# Gate F: Undergraduate Software Capstone

## Release a small original game on your engine

Gate F follows Volume 6 and combines its module projects with the engine from Volume 5. It is an engineering capstone, not a requirement to build a commercially large game. One coherent loop with deliberate constraints is preferable to many unfinished systems. The local multiplayer demonstration may be a separate executable or mode.

The project's original requirements are preserved. This expanded specification adds suggested milestones, evidence forms and an assessment rubric. The rubric is a teaching tool, not an accreditation standard. The sample theme and numerical scene budgets below are optional design suggestions, not mandatory performance promises.

## A bounded example: Signal Relay

A courier enters a small arena, activates three signal stations and reaches an exit while avoiding a patrolling hazard. A two-link antenna animates when a station activates. Activation creates a short audio cue and a visual message. The hazard uses inspectable patrol/investigate behavior; a blocked route produces a visible decision trace. The player can pause, remap an action, change music/effects levels and save progress.

The loop has a beginning, a meaningful decision, a success/failure condition and a restart. Use primitive geometry and original generated sounds where practical. The student may choose another original loop, including a tiny RTS scenario, provided all required subsystem evidence remains present. A five-minute loop with a handful of entities is sufficient scope; these are scope suggestions, not a minimum content length.

A separate two-client grid demonstration exercises network authority and recovery. Keeping networking outside the main game is explicitly allowed; this avoids turning the capstone into a public-service operations project. No public server or paid cloud account is required.

## Required system contract

Define units, world axes, transform multiplication order, fixed timestep, tick identity and entity generations. Simulation owns authoritative positions and gameplay state. Presentation interpolates without secretly changing physics. AI reads a coherent state snapshot and produces bounded intentions. Audio consumes bounded commands and cannot load files or take ownership of arbitrary gameplay objects during its callback.

Document the serialization schema and version. Distinguish replay state, persistent save state and presentation state. They overlap but are not interchangeable. A rollback snapshot needs every authoritative value necessary to reproduce the simulation, including RNG state; a presentation deduplication ledger must retain knowledge of already emitted one-shots. Save settings and progress through a tested replacement/recovery path.

Treat input, assets, mods and network messages as boundaries. Validate identifiers, versions, counts, ranges and ownership before allocation or mutation. No arbitrary code mods or public account service are required. When an external library is used, pin its revision and retain license/provenance information.

## Acceptance matrix

| Area | Required demonstration | Evidence |
|---|---|---|
| Gameplay | One complete original loop, success/failure and restart | Reviewer task record and source |
| Physics | Collision, one response case, fast-object test and finite state | Oracle/invariant tests and traces |
| Motion | Animation or articulated hierarchy with clear ownership | Bind/transform tests and interruption trace |
| Audio | Bounded native/platform path and category controls | Work caps, device/stress observations |
| AI | Inspectable decision and blocked/failure recovery | Scenario seeds, trace and cost data |
| Multiplayer | Local two-client fault/replay demonstration | Process topology and transport fault logs |
| Settings/save | Remapping, persistence and interrupted-save handling | Reload/recovery tests |
| Accessibility | Readable tested UI, pseudo-localization, critical visual audio alternative | Captures and observed muted task |
| Packaging | Clean-folder launch, versioned assets and provenance | Build/run record and manifest |
| Performance | Declared machine/budget, raw distributions and memory method | Repeated benchmark samples |
| Review | Independent loop completion, bug reproduction and live change | Named/date-stamped review record |

The full Module 30.3 data-only mod laboratory remains required even when the final game does not expose a mod interface. When mods are exposed in the release, the malicious/oversized mod tests become release-blocking. Do not claim unused or untested backend support merely because an abstraction exists.

## Milestone 1: scope and vertical slice

Write a one-page game specification and a single-scene budget. List the target platform, rendering route, input devices and audio route. Declare what is out of scope: for example, matchmaking over the internet, user-generated scripts, large worlds and commercial art. Identify the complete first playable loop before adding optional systems.

Build a clean checkout of the existing engine and verify its Gate E evidence still applies to this branch. Add only enough geometry/input to walk through the loop. Save a reproducible scene and input tape. The milestone passes when a reviewer can explain the intended objective and the engine runs without undocumented local files.

## Milestone 2: simulation and motion

Integrate conservative collision, explicit contact-normal convention and an appropriate response model. Include analytic or invariant checks and a fast-projectile regression. Add the articulated object or animation state machine. Specify root-motion ownership and cancellation behavior.

Capture authoritative state before/after key ticks. Change presentation rate without changing the input tape and verify the declared simulation behavior. Run the maintained physics-library comparison required by Module 26, retaining its assumptions and limits rather than claiming identical algorithms.

This milestone passes when a numerical defect produces an actionable first-failure trace, not merely a screenshot of a broken scene. Fix invalid-state propagation before layering AI and sound on it.

## Milestone 3: feedback, AI and persistence

Add the real platform audio path, bounded resources and semantic event identities. Complete the critical cue without sound, not just with a visible icon. Integrate the inspectable agent, bounded replanning/search and cancellation cleanup. Run the synthetic learning exercise even when the final game uses only rules/search.

Implement settings, remapping and save/load. Test a changed binding in actual gameplay and every relevant prompt. Interrupt a save before replacement and verify recovery. An exception-injection test is useful, but it must not be described as power-loss durability certification.

This milestone passes when the loop remains understandable under audio mute, missing optional devices and one intentionally blocked AI plan. Record usability findings as observations rather than assuming every player has the same needs.

## Milestone 4: network and service evidence

Connect a local authority and two clients through the declared process/socket adapter. The supplied in-process simulator is a starting reference, not the completed transport test. Run delay, jitter, loss, duplication and reordering, including input gaps and stale snapshots. Clearly label any reliable drain used to check eventual convergence.

Demonstrate interpolation and missing-baseline recovery, relevance filtering before serialization, and rollback or lockstep with a supported history/window policy. Show that one-shot audio and committed rewards are not duplicated by replay/retry.

Run registry/result-store tests with failure before commit, failure after commit before response, exact retry, conflicting retry and service restart. Synthetic principals are adequate for the teaching model but must be labeled as such. Do not deploy this local model as production authentication.

## Milestone 5: release candidate and review

Freeze the candidate. Record build mode, dependencies, scene seed, resolution and command line. Generate the distribution in a clean folder. Test both declared window sizes, remapping, pseudo-localized strings, muted critical task, controller absence and exit during loading. When the project supports a controller, test disconnect behavior rather than just successful startup with the device present.

Measure multiple runs. Preserve raw frame samples and peak-memory methodology. Separate CPU/GPU timing when a GPU backend is tested, or label software-renderer measurements accurately. Identify one measured bottleneck, attempt a justified optimization and report before/after results with variability and unchanged correctness. An inconclusive or negative result is acceptable; inventing a performance gain is not.

Arrange the independent review only after the candidate and evidence are ready. The reviewer must follow the launch instructions, finish a loop, reproduce a bug regression and observe a small live change. Record actual identity/date and what was observed. Empty templates and the rubric script are not completed review evidence.

## Required failure matrix

| Scenario | Required safe outcome |
|---|---|
| Save interrupted before replacement | Existing valid save remains readable or documented recovery succeeds |
| Unknown save/schema version | Clear refusal or tested migration; no silent corruption |
| Missing/invalid mod or asset | Bounded rejection and useful diagnostic, not arbitrary allocation |
| Device absent/disconnected | Defined fallback, pause or clear unsupported-state message |
| Exit during loading | Ownership-safe cancellation and shutdown |
| AI target becomes unreachable | Bounded failure/replan with an inspectable reason |
| Missing/reordered network input | No false contiguous ACK; documented recovery |
| Snapshot baseline missing | Full-state recovery; no application to the wrong baseline |
| Correction older than rollback history | Explicit resync/rejection policy |
| Duplicate committed result | Exactly one reward under idempotent retry |
| NaN/invalid simulation state | First-failure capture and controlled stop/recovery |
| Update interrupted before activation | At least one known-valid release remains selected |

Add project-specific failures when the loop has more boundaries. Do not mark an irrelevant test passed; mark it not applicable with the reason and confirm the original lesson requirement is still fulfilled elsewhere.

## Evidence directory and provenance

Create a release-evidence directory with subfolders for environment/build, scenes/inputs, correctness, physics comparison, audio, AI, network/service, accessibility, performance, packaging and review. Use stable filenames and short commands. Every claim in the acceptance matrix should name a file, test or observation.

Keep package reference results separate. The included 98 contract cases, randomized oracle suite, native offline harness and companion experiments describe these supplied implementations on their tested environment. They do not establish the student’s engine correctness, target-machine performance, platform audio, socket adapter or independent

review.

Include asset origin, license/permission and any transformations for every shipped asset. Include third-party library revisions and licensing notices. Record whether any model/dataset is synthetic, generated or externally sourced and which outputs were actually evaluated.

## Assessment rubric

| Category | Maximum | Full-credit evidence |
|---|---|---|
| Gameplay | 20 | Finished coherent loop, clear objective, valid restart and observed completion |
| Systems | 20 | Correct boundaries for physi cs/motion/audio/AI/network, explained tradeoffs |
| Correctness | 20 | Invariants, failure recovery, replay, malformed input and regression evidence |
| Production | 15 | Compatibility, settings, accessibility observations, clean package/provenance |
| Performance | 10 | Repeated raw measurements, bottleneck analysis and uncertainty |
| Documentation | 10 | Reproducible build, readable contracts and measured postmortem |
| Oral review | 5 | Independent explanation and successful live change/test |

A suggested passing threshold is 80/100, with all mandatory requirements met. Award partial points for correct but incomplete evidence and zero for unsupported claims. These are expanded-course assessment rules, not externally validated grading standards.

Mandatory blockers include a failed clean launch, unsafe/broken save recovery, unresolved critical defects, missing independent loop completion, missing live change, or missing required network demonstration. Any other unmet mandatory acceptance condition is also a blocker even when the convenience evaluator lacks a separate boolean field for it. A total of 100 does not override a blocker. The Lesson 30.4 evaluator only checks supplied scores/flags; it cannot verify the claims or approve the project.

## Oral review prompts

Explain one actual contact impulse using units and signs. Identify which state is restored before a rollback tick. Show how the callback avoids loading and unbounded command work. Use the AI trace to explain a failure rather than describing intended behavior. Derive one interpolation value from timestamps. Explain how a lost result response avoids a duplicate reward. Defend the benchmark boundary and one rejected optimization. Show how a remapped binding reaches both gameplay and its prompt. Finally, make a small live change, predict its effect and rerun the relevant regression.

The review is not a memory contest. Referencing the project’s own documentation is reasonable, but the learner must demonstrate understanding and independent modification. A reviewer may request a changed input or counterexample to distinguish understanding from copied code.

## Postmortem structure

State the original scope and the actual delivered loop. Compare three predictions with measured outcomes, including one that was wrong. Explain the most significant boundary/ownership bug and the test that now prevents it. Present the optimization experiment, including unchanged correctness and variance. Summarize human observations and fixes. Separate supported features, known limitations, deferred validation and removed scope. End with one bounded next improvement and the evidence needed to judge it.

A successful Gate F release is small, understandable, testable and honestly documented. It is the foundation for the subsequent hardware and graduate specialization volumes, not a claim of commercial readiness or a substitute for an accredited degree.